# Introducción a VSCode y GitHub

## EIE390 - Robótica e Inteligencia Artificial

Profesor: Guillermo Cid Ampuero

PUCV

# ¿Qué veremos hoy?

- VSCode
- GitHub

# VSCode

Visual Studio Code (VS Code) es un editor de código fuente ligero, gratuito y de código abierto, desarrollado por Microsoft. Es multiplataforma (Windows, macOS, Linux) y se destaca por su alta capacidad de personalización, soporte para múltiples lenguajes de programación, depuración, control de Git integrado y una enorme biblioteca de extensiones.

Figure: VSCode

# Instalar VSCode

Desde la pagina de VSCode (Link) se debe descargar según su computador.

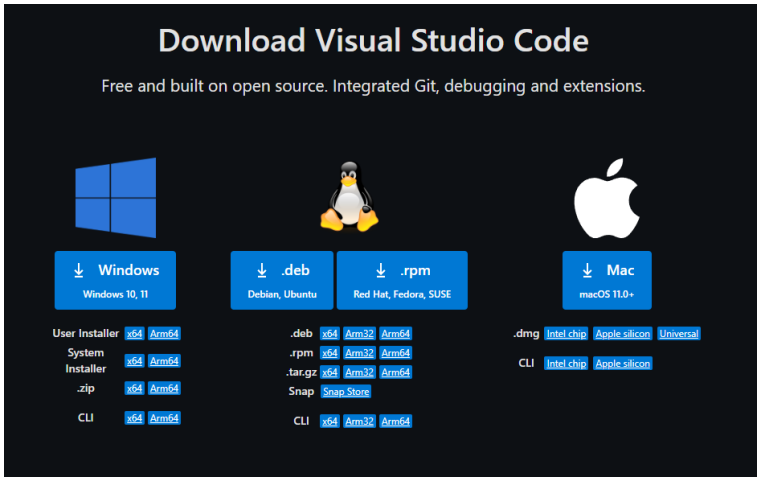



Figure: Descargar VSCode

A continuación los links con tutoriales de descarga:

- Window
- Ubuntu
- Mac

# GitHub

GitHub es una plataforma en la nube diseñada para desarrolladores que permite almacenar, gestionar y colaborar en proyectos de código fuente utilizando el sistema de control de versiones Git. Actúa como un repositorio central, facilitando el trabajo en equipo, el seguimiento de cambios y la colaboración en software de código abierto y privado.

Figure: GitHub

# GitHub

Al igual que VSCode , GitHub debe ser instalado, dependiendo de su plataforma.

- Window
- Ubuntu
- Mac

# Crear cuenta en GitHub

Para realizar su cuenta en GitHub deben ingresar en la pagina de GitHub (Link) y crearse una cuenta con su correo.

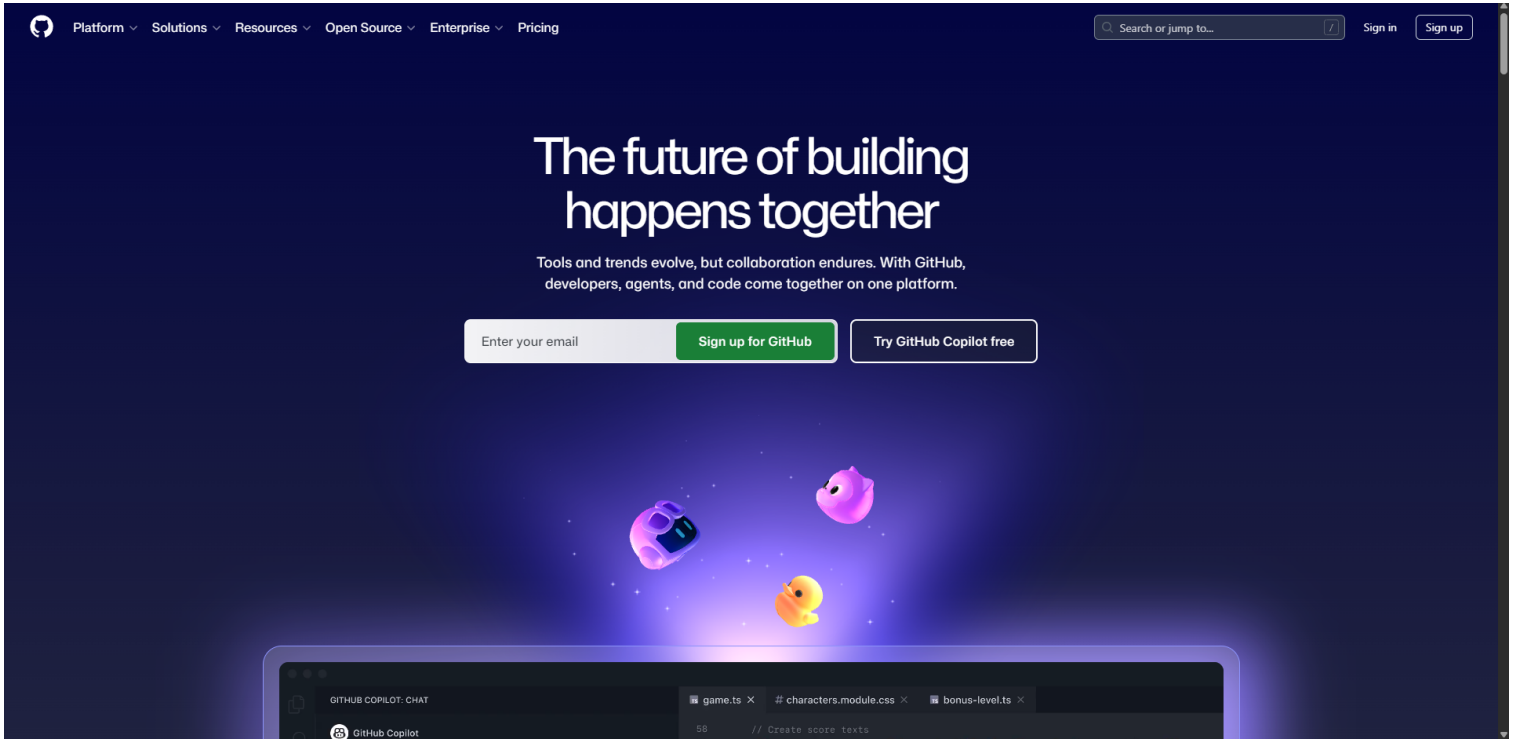



Figure: GitHub

# Repositorio de Github

Un repositorio en GitHub (o "repo") es un espacio de almacenamiento digital centralizado donde se aloja un proyecto, incluyendo todos sus archivos, el código fuente y el historial de cambios de cada uno. Actúa como una carpeta de proyecto en la nube, ideal para colaboración, control de versiones y gestión, permitiendo acceder a versiones anteriores y trabajar en equipo. Este puede ser privado o publico, puede ser modificado por personas que tengan acceso a el, y cada persona puede tener su nombre propio, lo que permite saber quien a modificado el repositorio.

# Crear repositorio de Github

Para crear un repositorio en GitHub, se hace desde la misma pagina de GitHub, en el botón **New**.

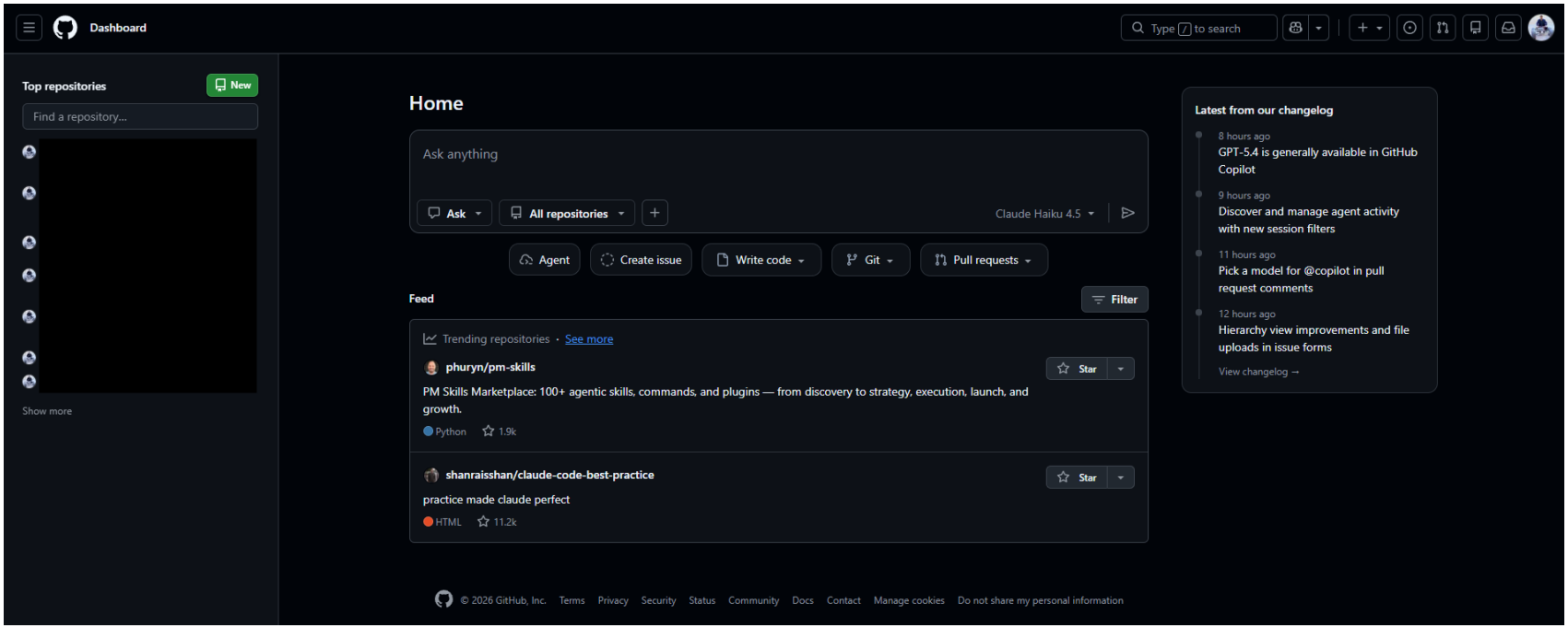
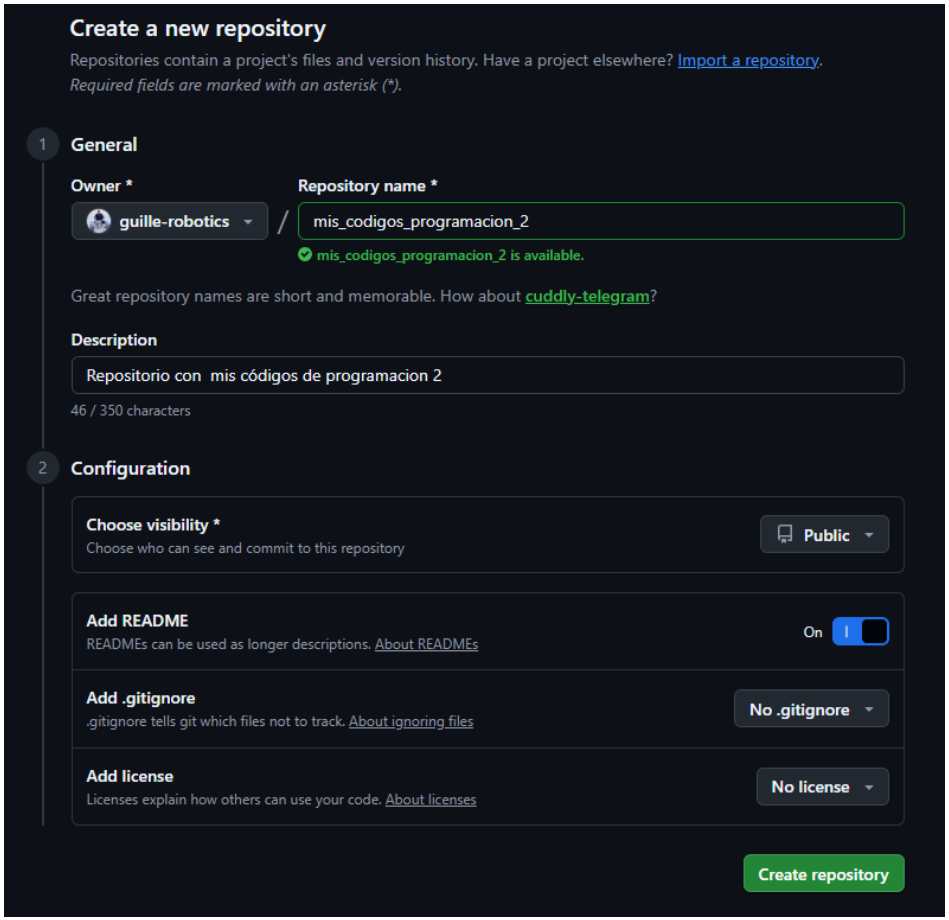



Figure: Página inicial para crear repositorio

# Crear repositorio de Github

Luego de dar a **New** aparecera la siguiente pagina, aca le daran el nombre a su repositorio y es recomendable habilitar la opción de **Add README**. Luego al botón *Create repository*.



# Crear repositorio de Github

## README

Un archivo `README` en GitHub (`README.md`) es el documento principal de presentación y documentación de un repositorio, generalmente escrito en formato *Markdown*.
Funciona como la "puerta de entrada" al proyecto, detallando qué hace, por qué es útil, cómo se instala y cómo se utiliza. Es la primera información que ven los visitantes al ingresar al repositorio.

# Crear repositorio de Github

Luego de pulsar el botón *Create repository*, obtendremos la siguiente página, la cual nos da recomendaciones de como usar nuestro repositorio o nos mostrará directamente nuestro repositorio.

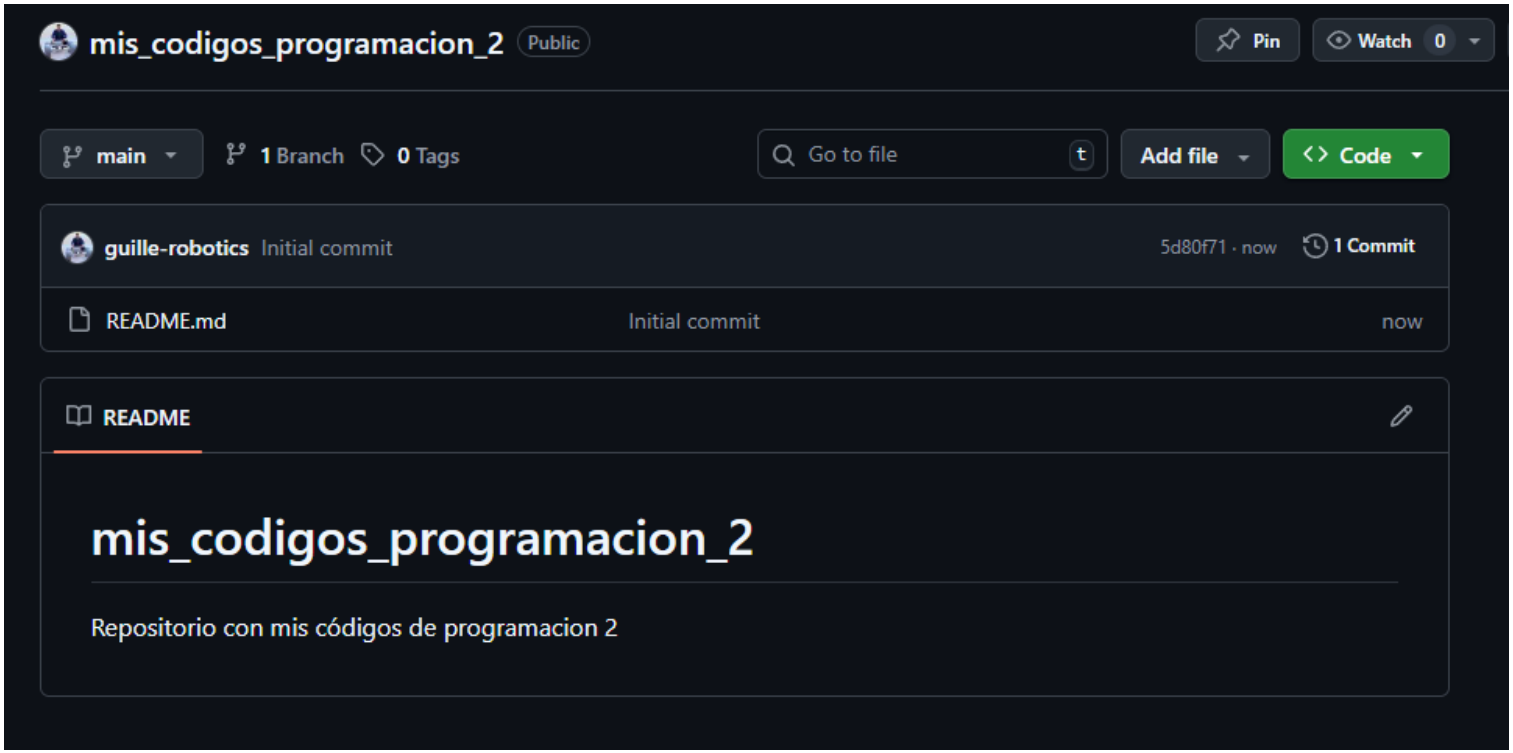



Figure: Nuestro repositorio.

# Trabajar con nuestro repositorio de Github

La manera mas sencilla de trabajar con nuestro repostorio es directamente desde nuestra PC. Para eso, copiamos el link de nuestro repositorio.

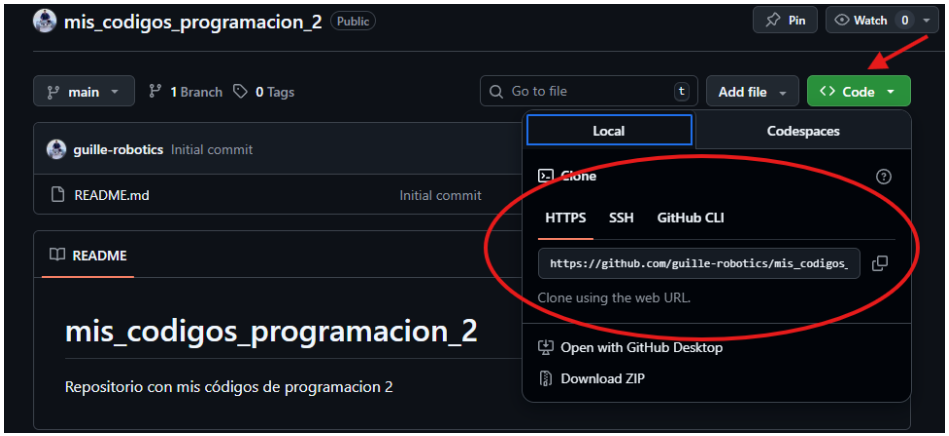



Figure: Link de nuestro repositorio

Y usamos `git clone <link del repositorio>` para clonar el repositorio en nuestro PC.

# Trabajar con nuestro repositorio de Github

El comando `git clone <link del repositorio>` tiene variaciones:

```
1 git clone -b nombre\_rama <link del repositorio>
2 git clone --branch nombre\_rama --single-branch <link del
    repositorio>
```

### Rama

Un branch (rama) en GitHub es una línea de trabajo independiente dentro de un repositorio GitHub.

# Trabajar con nuestro repositorio de Github

Cuando uno va a trabajar con git , es importante decirle al computador quien esta trabajando, por lo que en un punto del trabajo GitHub te solicitará.

```
1 git config --global user.name "Nombre Apellido"
2 git config --global user.email "correo@correo.com"
```

Por lo que es mejor hacerlo de manera inmediata.

# Trabajar con nuestro repositorio de Github

Luego desde nuestro PC, en la carpeta donde clonamos nuestro repositorio, la abrimos con un terminal y ejecutamos desde la terminal. `code .`

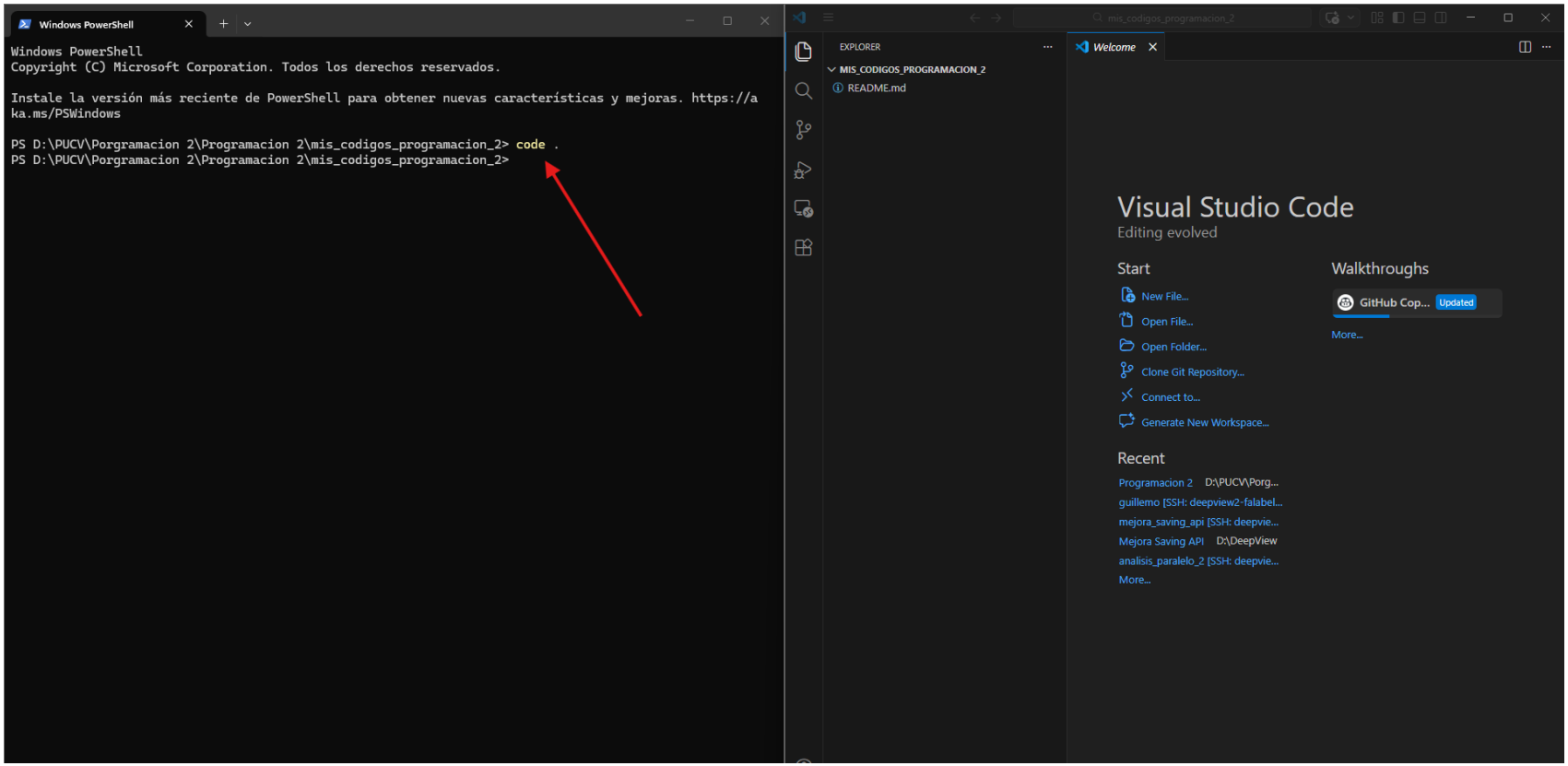



Figure: Abrir nuestro repositorio

# Trabajar con nuestro repositorio de Github

Ahora procederemos a crear nuestro primer código y subirlo a nuestro repositorio de Github. Para ello seguiremos los siguientes pasos:

- Crear un archivo de código llamado `primer_codigo.py`.
- Luego ejecutaremos los siguientes comandos de Git para subir el archivo al repositorio.
  - `git branch` // *Verifica en que rama estas*
  - `git status` // *Verifica el estado del repositorio*
  - `git add .` // *Agrega los cambios al área de preparación*
  - `git commit -m "Primer codigo"` // *Guarda el cambio en el historial*
  - `git push` // *Envía los cambios a Github*

### Commit

Un commit en Git/GitHub es una "instantánea" o punto de guardado de tus archivos en un momento específico

# Trabajar con nuestro repositorio de GitHub

Los comandos anteriores se utilizan cuando trabajamos con un repositorio que ya existe. Sin embargo, muchas veces tendremos un proyecto con varios archivos y carpetas en nuestro computador y querremos subirlo a GitHub.

En ese caso, primero debemos crear el repositorio en GitHub y luego ejecutar los siguientes comandos desde la carpeta principal del proyecto:

```
1 git init
2 git add .
3 git commit -m "Primer commit"
4 git remote add origin URL
5 git push -u origin main
```

Para subir nuevas modificaciones al repositorio utilizamos:

```
1 git status
2 git add .
3 git commit -m "Nuevo commit"
4 git push
```

# Otros comandos útiles de Git

Durante el desarrollo existen otros comandos que nos ayudan a revisar y gestionar los cambios en nuestro repositorio.

```
1 git status          # Ver estado del repositorio
2 git diff            # Ver diferencias en los archivos
    modificados
3 git restore         # Descartar cambios en un archivo
4 git branch          # Ver las ramas del repositorio
5 git switch rama     # Cambiar de rama
6 git pull            # Descargar cambios del repositorio remoto
```

# Ejemplo : Uso de `git push` y `git pull`

Supongamos que tenemos el mismo repositorio clonado en dos computadores:

- Un PC personal
- Una Raspberry Pi

**1. Realizamos cambios en nuestro PC y los subimos a GitHub:**

```
1 git add .
2 git commit -m "Agrego nuevas funciones"
3 git push
```

**2. Luego, en la Raspberry Pi descargamos los cambios:**

```
1 git pull
```

**Idea clave:**

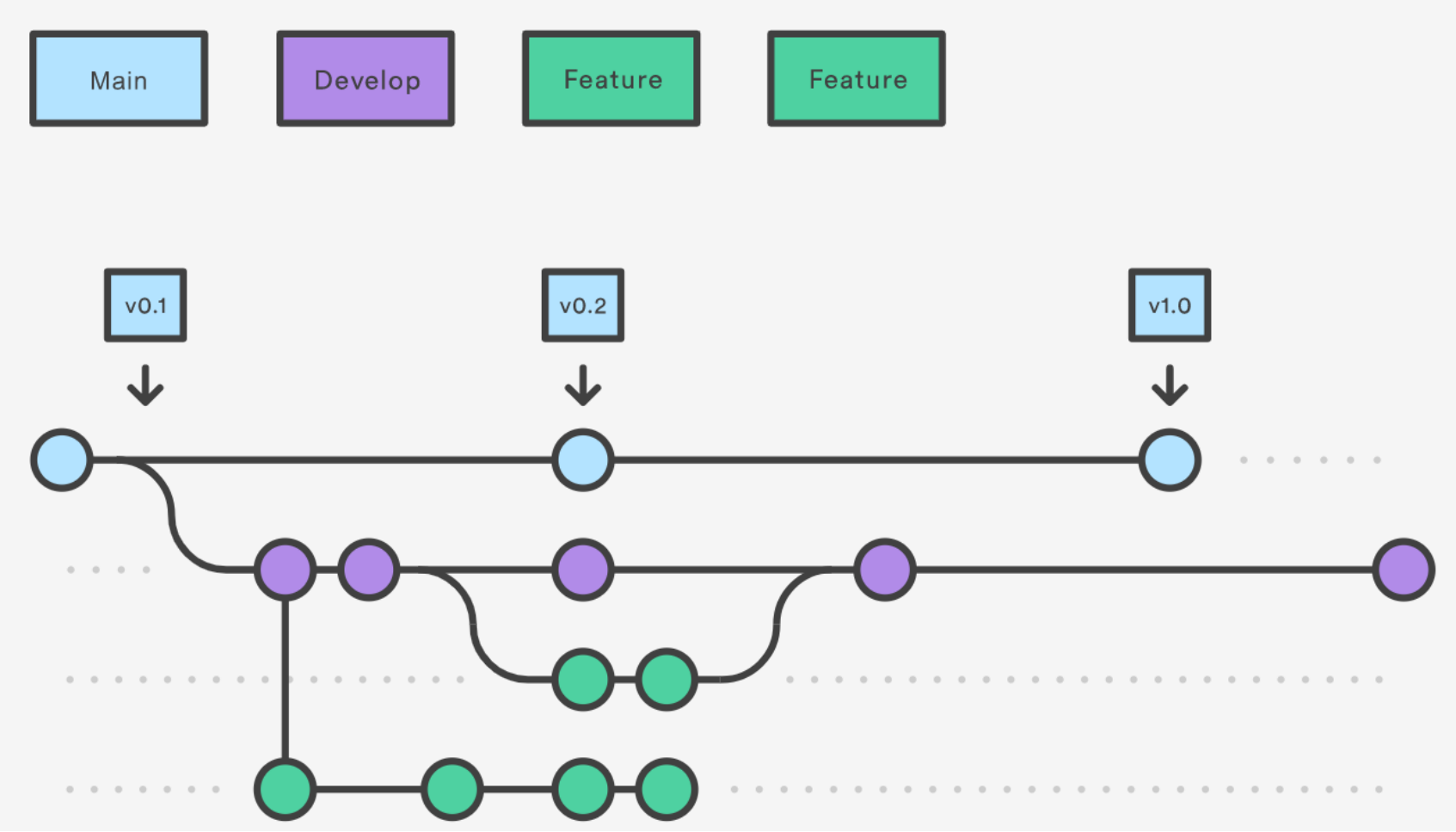
- `git push` → subir cambios a GitHub
- `git pull` → descargar cambios desde GitHub

# Ejemplo: Clonar repositorio

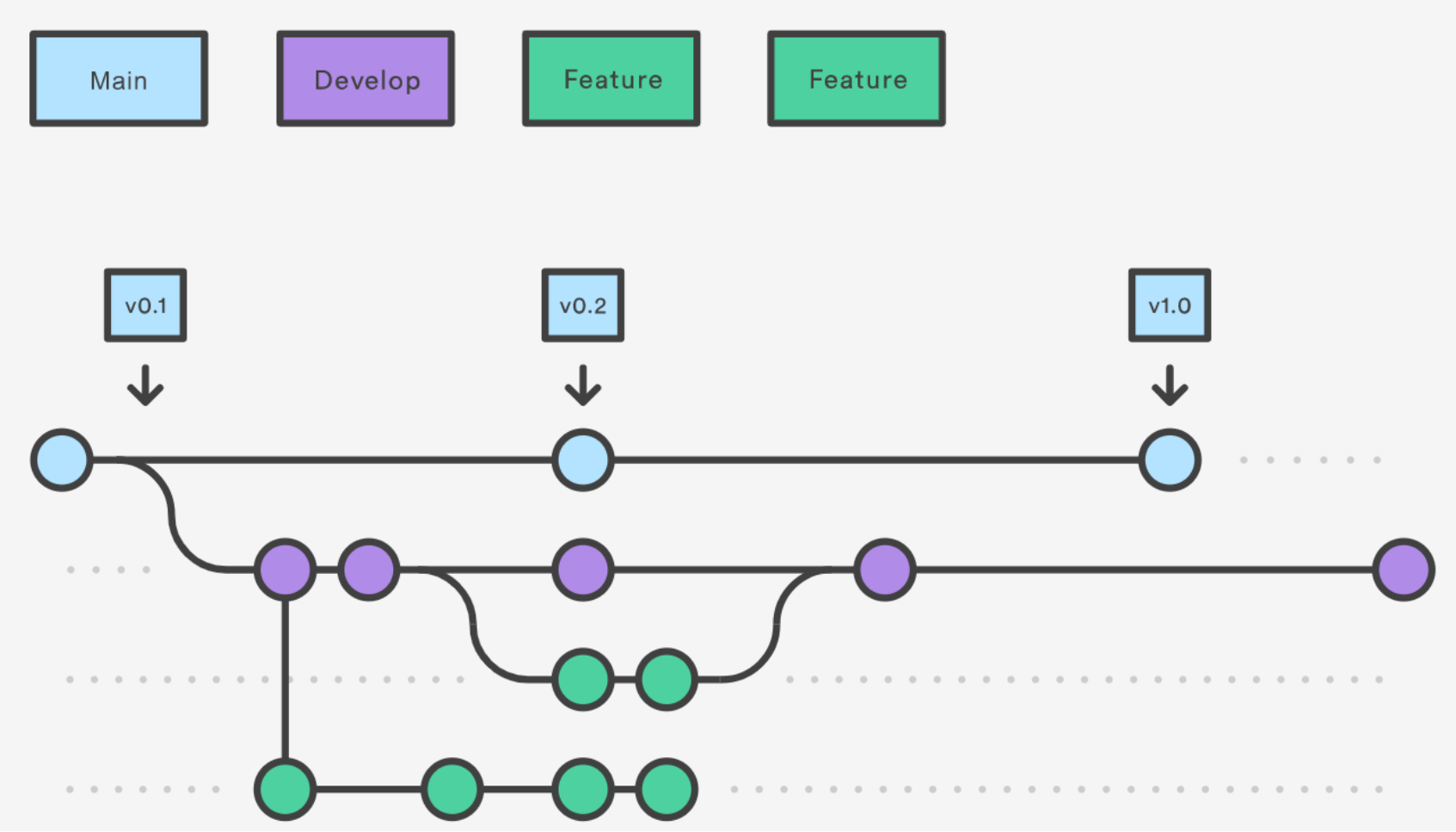
Mucha veces se te solicitar que clones un repositorio, pero que trabajes en la rama **develop**, por lo que deberas realizar lo siguiente:

```
1 git clone URL_DEL_REPO
2 cd repo
3 git branch              # ver ramas
4 git switch -c develop # crear develop
5 git push -u origin develop
```

# Estructura de ramas en Git



# Estructura de ramas en Git



# Estructura de ramas en Git

- **Main**: contiene la versión estable del proyecto (versiones publicadas como v0.1, v0.2, v1.0).
- **Develop**: rama donde se integran los cambios antes de pasar a la versión estable.
- **Feature**: ramas temporales donde se desarrollan nuevas funcionalidades.
- Cuando una funcionalidad está lista, se **fusiona (merge)** nuevamente a `develop` y luego a `main`.

# Ejemplo de uso de ramas en una empresa

Supongamos que una empresa está desarrollando una aplicación web.

- **main**: contiene la versión estable del sistema (la que usan los clientes).
- **develop**: rama donde se integran nuevas funcionalidades antes de liberarlas.
- **feature_login**: un desarrollador crea esta rama para implementar el sistema de login.

# Ejemplo de uso de ramas en una empresa

**Paso 0: Ver en qué rama estamos trabajando**

```
1 git branch
```

El símbolo $*$ indica la rama actual.

**Paso 1: Integrar la funcionalidad a develop**

```
1 git switch develop
2 git merge feature_login
```

Ahora la funcionalidad de login está integrada en `develop`.

**Paso 2: Liberar una nueva versión del sistema**

```
1 git switch main
2 git merge develop
```

Esto mueve todos los cambios desde `develop` hacia `main`.

**Flujo típico en empresas:**

`feature → develop → main`

# Ejemplo

```
1 git branch -> ver donde estoy parado, * representa en donde
    estoy
2 git switch -c pedro -> creo la rama pedro y me cambio a la
    rama
3 git branch -> para confirmar que cambie a pedro
4 git add . -> añado los cambios que realizo pedro
5 git commit -m "Codigos de Pedro" -> anuncio lo que pedro va
    a subir"
6 "Seguramente aca cuando es primera vez le solicitaran su
    email y usuario"
7 git push -u origin pedro -> subo los cambios de pedro
8 ## Quiero subir los cambios de pedro a main##
9 git switch main -> me cambio a main
10 git branch -> reviso si me cambie
11 git merge pedro -> le digo git los cambios de pedro llevalos
     a main
12 git push -> sube los cambios a main
```